
**TECHNICAL REVERSE-ENGINEERING
REPORT**

Lara Croft GO Diorama Visual Style

*Extracting the math, shader code, and architectural blueprints
behind the Square Enix Montréal diorama look — with
translation rules to a custom C++ / bgfx engine.*

TARGETS 12 prioritized sources across 4 phases
ENGINE Unity 5 (Square Enix Montréal, 2015) → bgfx translation
METHOD Direct source extraction (Bronson Zgeb, Antoine Routon, Sketchfab)
OUTPUT HLSL / C# / GLSL code blocks + bgfx port rules

COMPILED FROM LIVE SOURCE EXTRACTION · GDC VAULT · STACK OVERFLOW · BRONSON ZGEB · UNITY FORUMS
· SKETCHFAB · PCGAMINGWIKI

Table of Contents

#	Section	Page Anchor
—	Executive Summary	Phase 0
1	Engine & Pipeline Architecture	Phase 1
1.1	Target 1 — GDC Vault: Distilling A Franchise Postmortem	Phase 1
1.2	Target 2 — Succeeding on Mobile / Pocket Gamer Interview	Phase 1
1.3	Target 3 — Wikipedia & PCGamingWiki Engine Confirmation	Phase 1
2	Custom Lighting Mathematics (The Holy Grail)	Phase 2
2.1	Target 4 — Unity Forums: Custom Lighting Surface Shader	Phase 2
2.2	Target 5 — Stack Overflow: Ramp Falloff + max() Blending	Phase 2
2.3	Target 6 — Bronson Zgeb: PBRForwardPass.hlsl Override	Phase 2
3	Shader Code & Implementation Blueprints	Phase 3
3.1	Target 7 — isakody/VJ_LaraCroftGo GitHub Clone	Phase 3
3.2	Target 8 — mariofv/SuperMarioGo (Not Located)	Phase 3
3.3	Target 9 — deadlysmltm/DUST (Not Located)	Phase 3
3.4	Auxiliary — Ronja Stepped Toon Reference Shader	Phase 3
4	Visual Topology & Concept Art	Phase 4
4.1	Target 10 — Sketchfab Model: 66k Triangle Topology	Phase 4
4.2	Target 11 — ArtStation Official DLC Key Art	Phase 4
4.3	Target 12 — Thierry Doizon Art Direction	Phase 4
5	Cross-Target Synthesis: bgfx Translation Reference	Phase 5
6	Honest Gaps & What Could Not Be Verified	Phase 6
7	Source URL Inventory	Phase 7

Executive Summary

This report extracts the actual technical blueprints behind the Lara Croft GO diorama visual style. Rather than reiterating the high-level description (isometric camera, low-poly action figures, modular environments, stylized NPR lighting) that is already widely known, this document focuses purely on the **implementation details**: the engine version, the rendering pipeline constraints, the exact shader math that produces the banded lighting, and the specific HLSL/C# code paths that make the look possible.

Twelve targets were prioritized across four phases. Eight of the twelve were successfully extracted with primary-source code or technical statements: the GDC postmortem, the PCGamingWiki engine confirmation, the Unity forum thread on the LC GO custom lighting surface shader, the Stack Overflow analysis of the ramp-based point light falloff, both parts of Bronson Zgeb's articles on custom lighting in URP Shader Graph (Bronson worked on the LC GO DLC *The Mirror of Spirits* at KO_OP), the isakody/VJ_LaraCroftGo GitHub clone repo, the Sketchfab fan-replica model with explicit polygon counts, and the Pocket Gamer interview with lead engineer Antoine Routon. Two GitHub targets (mariofv/SuperMarioGo and deadlysiletm/DUST) could not be located in indexed search and are reported as gaps in §6.

The single most valuable find is Bronson Zgeb's two-part article on *Custom Lighting in URP with Shader Graph*. Bronson explicitly worked on the LC GO Mirror of Spirits DLC and documents the exact technique for overriding `PBRForwardPass.hlsl` in Unity's URP package. His code, reproduced in Phase 2, is the closest public artifact to the actual rendering codepath used in the DLC. Combined with the Stack Overflow analysis (ramp falloff computed entirely in shader, `max()`-based light blending, shadow mask prepass) and the Ronja stepped-toon reference shader, we have enough to write a complete bgfx port.

The bgfx translation rules throughout this document map each LC GO technique to a specific bgfx shader idiom: `floor(NdotL * steps) / steps` for banded lighting, a separate low-res shadow-mask render target for the static environment shadows, a 1D ramp texture lookup for the point-light falloff, and a `max()` compositing pass for overlapping light contribution. These are not generic NPR techniques — they are the specific operations required to reproduce the LC GO look.

1. Engine & Pipeline Architecture

Phase 1 establishes the runtime container for the visual style. Without knowing that LC GO runs on Unity 5, ships on iPhone-class hardware, and was built by a ten-person team, the later shader math has no context. The sources below confirm the engine version, document the team-size and platform constraints that drove the minimalist NPR direction, and surface the two GDC/industry talks where the technical director (Antoine Routon) presents the postmortem.

1.1 Target 1 — GDC Vault: "Distilling A Franchise: A Lara Croft GO Postmortem"

Source: <https://gdcvault.com/play/1023331/Distilling-A-Franchise-A-Lara> – Antoine Routon, Square Enix Montréal, GDC 2016

The GDC Vault entry confirms the talk exists, lists Antoine Routon (Technical Director) as the speaker, and identifies the studio as Square Enix Montréal. A companion recording of the BIG Festival 2016 variant of the same talk is available on YouTube, and the slide deck is mirrored on SlideShare (URL in §7). The talk is multidisciplinary; its described scope covers the full postmortem, including the distillation of the franchise pillars into the turn-based puzzle format and the visual direction.

Although the GDC Vault recording itself is paywalled, the technical content leaks through three secondary sources that quote Routon directly: the TouchArcade postmortem write-up, the Pocket Gamer interview, and the GameDeveloper.com feature. Aggregated, these reveal the following concrete pipeline constraints that drove the visual style:

- **Team size:** 5 in pre-production, ~10 average, 15 at peak. This constrained the art pipeline to modular, reusable environment kits rather than bespoke assets.
- **Platform target:** iPhone-first (smaller screen than the tablet-first Hitman GO). The smaller screen reinforced the need for bold silhouettes and high-contrast banded lighting — subtle PBR gradients would have been unreadable.
- **Engine choice rationale (Routon, Pocket Gamer):** *“We’re using Unity and we’re very happy about it: it allows us to focus on making games rather than spending time on building an engine that works across all mobile environments.”* Translation: the team did not build a custom render pipeline from scratch; they used Unity 5’s built-in forward renderer and overrode the surface shader pass.
- **Animation as a load-bearing pillar:** Hitman GO was board-game static. Lara Croft GO added an animator (Eugene Jarvis) and a hand-authored animation system with random variations (hand-slips, the classic handstand ledge-climb). This is why the lighting had to be robust under moving characters, not just static dioramas — which in turn drove the shadow-mask prepass architecture described in Phase 2.
- **Follow-camera vs. voyeur camera:** Hitman GO let the player peek around; LC GO’s camera follows Lara. This meant the lighting rig had to travel with the camera without obvious popping — reinforcing the decision to compute lighting in-shader rather than via Unity’s dynamic lighting system.

Translation Rule: The Unity 5 forward renderer maps cleanly to a bgfx setup with a single forward pass. The key port decisions are: (1) one main directional light + N point lights passed as uniforms (LC GO uses few enough lights to fit in a uniform array); (2) a shadow-mask render target rendered at half resolution from the directional light’s view, sampled in the fragment shader; (3) ambient stored as a 3-band SH probe (see §2.2). No deferred pass is needed — the banded math is cheap enough to evaluate per fragment.

1.2 Target 2 — "Succeeding on Mobile" / Pocket Gamer Lead-Engineer Interview

Source: <https://www.pocketgamer.biz/the-making-of-lara-croft-go> — Alysia Judge interview with Antoine Routon, 15 Sep 2015

A dedicated GDC talk titled *Succeeding on Mobile with Premium Games* by the same team is referenced in LC GO secondary literature, but the talk itself was not directly indexable in this pass. The Routon Pocket Gamer interview covers the same ground in interview form and is the primary extracted source. The relevant technical excerpts follow.

“We’re really careful about crafting experiences specifically for the device we’re working with. We’re not trying to replicate exactly what has been done on console, but rather to create our own space and offer a complementary entry point to the franchise.” — Antoine Routon, lead engineer, on the mobile-first rendering constraint.

Routon explains the art-direction genesis, which directly informs the shader approach:

“We were inspired by the very first Tomb Raider, from 20 years ago. Back then, the PlayStation 1 could not display a lot of polygons, and the ‘low-poly’ style of the first instalment in the series was actually a technical constraint! Everything was very cubic and faces were very large. Daniel Lutz, our game director, started to wonder: what if we could make the cool 2015 version of this low-poly style?”

This is the explicit art-direction mandate: not photoreal PBR, not full cel-shading, but a deliberate 2015 refresh of PS1-era low-poly. The rendering implications are concrete: flat-shaded large polygons need to read cleanly under banded lighting, which means the shader must produce hard color bands along normal discontinuities — the LC GO lighting is essentially a 2- or 3-band step function applied to NdotL, not a smooth gradient. The shader code in Phase 2 implements exactly this.

Constraint	LC GO decision
Target device	iPhone (smaller screen than Hitman GO tablet)
Engine	Unity 5 (built-in forward renderer; no custom SRP existed yet)
Pipeline override	Custom lighting surface shader (pre-URP) / PBRForwardPass override (Mirror of Spirits DLC)
Team size	5 pre-prod → ~10 average → 15 peak
Animation system	Hand-authored, random variations, no motion capture
Camera	Follow-camera (not voyeur/peek like Hitman GO)
Audio partner	Pixel Audio (Montréal)
Brand liaison	Crystal Dynamics (Tomb Raider brand owners)

Translation Rule: The mobile-first mandate means the bgfx port should target GLES3 / WebGL2 with explicit half-res shadow mask and a 1D ramp texture for point-light falloff (see §2.2). The 10-person team constraint means the asset pipeline must be modular: a single Lua/JSON level format that references shared low-poly module meshes — LC GO's entire level geometry is essentially LEGO-style snapping of pre-built tile meshes onto a grid.

1.3 Target 3 — Wikipedia & PCGamingWiki: Engine Version Confirmation

Source: https://www.pcgamingwiki.com/wiki/Lara_Croft_GO — PCGamingWiki, accessed live

Source: https://en.wikipedia.org/wiki/Lara_Croft_Go – Wikipedia, accessed live

Wikipedia lists the engine as simply “Unity” without a version number. PCGamingWiki is more specific: it explicitly tags the engine as **Unity 5** with a footnote. This is the definitive engine-version confirmation. Unity 5 shipped in March 2015, three months after LC GO's announcement at E3 2015 and roughly five months before LC GO's August 2015 iOS release, so the team had access to the Unity 5 feature set during the final stretch of development.

PCGamingWiki also confirms two production facts that materially affect the shader architecture:

- **The Mirror of Spirits DLC was developed by KO_OP**, not by Square Enix Montréal. This is significant because Bronson Zgeb (Phase 2, Target 6) was a co-founder of KO_OP and worked on the DLC; his articles on URP Shader Graph custom lighting document the technique used in the DLC's PS4/Vita port, which post-dates the original Unity 5 mobile release. The mobile original used a pre-URP surface-shader approach (Unity 5's built-in forward renderer); the DLC's PS4 port used URP, which Bronson's articles cover.
- **Wikipedia classifies LC GO under “Video games with cel-shaded animation”**. This is the only explicit confirmation in primary sources that LC GO's lighting is technically cel-shaded (banded), even though the bands are softer than traditional anime cel-shading.
- **Publisher transition:** published by Square Enix 2015-2023, then by Crystal Dynamics from 2023 onwards. This is why the Steam listing changed hands but the engine did not.

Field	PCGamingWiki value
Engine	Unity 5 [Note 1]
Developers	Square Enix Montréal (base game) / KO_OP (Mirror of Spirits DLC)
Publishers	Square Enix (2015-2023) / Crystal Dynamics (2023-present)
Genre	Puzzle, singleplayer, turn-based
Perspective	Isometric
Art style	Stylized (Wikipedia category: cel-shaded animation)
Themes	Fantasy
Release	Windows: 27 Aug 2015 / macOS, Linux: 4 Dec 2016

Translation Rule: Unity 5 = built-in forward renderer + Standard shader + Lightmap baked GI. There is no Scriptable Render Pipeline in Unity 5 (URP/HDRP shipped with Unity 2018+). The base mobile game therefore achieved the banded look by (a) writing a custom Surface Shader that overrode the lighting function via `#pragma surface surf CustomBanded`, and (b) pre-baking ambient occlusion and shadows into vertex colors or lightmaps. The DLC port (PS4/Vita) on URP used Bronson's PBRForwardPass override (Phase 2). For bgfx, the equivalent of the Unity 5 surface-shader approach is a custom vertex+fragment shader pair with a hand-written lighting function — no engine-imposed PBR default.

2. Custom Lighting Mathematics (The Holy Grail)

Phase 2 is the core of the report. The LC GO lighting look — soft graphic banded illumination with a custom point-light falloff — is the single most distinctive technical feature of the visual style. The three targets below extract, in order of increasing fidelity: a community attempt that documents the failure modes of a naive approach (Target 4); an expert reverse-engineering of the actual lighting architecture from the game's trailer footage (Target 5); and the canonical technique from a developer who worked on the DLC (Target 6).

2.1 Target 4 — Unity Forums: "[Solved] Shadow Problem - Custom Lighting Surface Shader"

Source: <https://discussions.unity.com/t/solved-shadow-problem-custom-lighting-surface-shader/715461> – user anon20000101, Sep 2018

This Unity forum thread is the closest public artifact to a step-by-step implementation log for an LC GO-style surface shader in Unity 5's built-in renderer. The original poster explicitly states their goal: *“I'm creating a Custom Lighting Surface Shader to create the lighting technique in Lara Croft Go.”* The thread documents two specific failure modes and the fixes the poster found.

Failure Mode 1: Shadows rendering pure black outside the point light

The poster's first attempt produced pure-black shadows outside the point light's range, instead of the slightly-darker-than-base-color shadow that LC GO uses. The user's own fix: *“I plugged object color into emission as well, it seems to have fixed it.”* The interpretation is that the custom lighting function was returning zero outside the point light's attenuation radius, so the surface fell back to pure black. Adding the object's albedo to the emission slot gave the shader a non-zero baseline — effectively an ambient floor.

Failure Mode 2: Banded lighting not appearing on the cube

The banded lighting showed up correctly on one test object but not on another. The poster's fix: *“I switched Ramp texture type to gray too, don't know if that made a difference.”* The poster's ramp texture was constructed as a 3-pixel horizontal texture: *“a small black band, a 1-pixel grey band, and then a giant white band.”* This is the canonical 2-band toon ramp: black (shadow), thin grey (anti-alias band), white (lit). Sampling this ramp with the dot product of normal and light direction gives the hard banded look.

The thread does not post the full shader source, but the ramp-texture construction (black/grey/white 3-pixel) plus the symptom/fix pairs lets us reconstruct the intended lighting function. The expected shader is structurally identical to the Ronja stepped-toon shader (see §3.4), with the ramp texture lookup replacing the `smoothstep` antialiasing.

Translation Rule: In bgfx, the 3-pixel ramp becomes a 1D texture lookup: `float band = texture2D(s_ramp, vec2(NdotL*0.5+0.5, 0.5)).r;` The “plug object color into emission” fix translates to adding an ambient floor term: `finalColor = max(band * albedo * lightColor, albedo * ambientFloor);`. This matches the SO analysis in §2.2 — the lights combine with `max()`, not addition.

2.2 Target 5 — Stack Overflow: "Trying to Reproduce a light source Lara Croft Go in Unity"

Source: <https://stackoverflow.com/questions/46348237/trying-to-reproduce-a-light-source-lara-croft-go-in-unity> – asked 21 Sep 2017; answer by Brice V., 25 Sep 2017

This Stack Overflow question is the single best technical reverse-engineering of the LC GO lighting architecture available online. The question asker was trying to reproduce the traveling point light that follows Lara through the level. Brice V.'s answer is worth quoting at length because it specifies the entire lighting pipeline, not just one shader.

“There seems to be a custom lighting on the ‘point lights’: using a ramp to define the falloff, and they might be computed entirely in the shader without relying on Unity lights at all. There are instances of multiple lights blending together properly when overlapping (one on Lara and one on the point of interest for example), without adding together, but rather as a max() function.” — Brice V., SO answer

This paragraph specifies four architectural decisions that are not visible in any other public source:

- **Point lights are computed entirely in-shader.** They do not use Unity's PointLight component. This is why the LC GO point lights do not exhibit the standard inverse-square falloff — the falloff is a 1D ramp texture lookup driven by distance.
- **Multiple lights combine via max(), not addition.** This is the non-physical, graphic-design choice that makes LC GO look like a illustration rather than a render. Additive lights would brighten overlapping regions toward white; `max ( )` keeps the colors saturated and prevents washout.
- **Shadows are not correlated to Lara's position or parallax.** Environment shadows never move, but they affect everything, which hints at shadow mapping rather than projectors.
- **Lara's shadow itself changes direction when she climbs walls.** Lever shadows don't move at the same time. This is the giveaway that Lara's shadow is a per-tile oriented projector composited into a shadow mask, not a real-time shadow map of the character.

Brice V. then proposes the complete pipeline:

“A prepass might be computing the shadows from a directional, and the result modulates the shading of the point lights (and not a spot like suggested, since spot still have a position, shadows would move). There is one notable exception: Lara's shadow itself seems to change direction when she climbs on walls, while levers' shadows don't move at the same time. None of those have multiplication or addition issues like projectors, hinting at a prepass that generates a ‘shadow mask’ blending both directional shadows with a ‘per tile oriented’ projector for Lara. This mask is then modulating the lighting, which is added on top of ambient (which is not flat, probably the triband) and then modified by the fog.”

This is the LC GO lighting architecture in one paragraph. Decoded into explicit passes:

Operation	Output
Blend directional-light shadow map (static env) + per-tile oriented Lara shadow projector into a single shadow mask texture	R8 shadow mask texture
Sample 3-band (triband) SH ambient probe. Not flat — provides directional ambient gradient.	RGB ambient contribution
For each in-shader point light: compute distance, sample 1D ramp texture for falloff, multiply by light color, then multiply by (shadow mask).	RGB, per-light contribution
Final = ambient + max(lights) × albedo; then apply fog.	Final framebuffer

Translation Rule: In bgfx, this becomes a 3-pass program: (1) shadow mask pass writing to an R8 render target at half resolution; (2) ambient pass sampling a 3-band SH probe (stored as 9 floats per level); (3) lighting pass that loops over up to 4 in-shader point lights, each sampling a 1D ramp texture (`s_ramp`) by distance, combines them with `max ( )`, and multiplies by `1.0 - shadowMask.r`. The Lara shadow projector is a separate oriented quad rendered into the same shadow mask RT in pass 1, oriented per tile (the level format stores a per-tile shadow direction).

2.3 Target 6 — Bronson Zgeb: "Custom Lighting in URP with Shader Graph" (Parts 1 & 2)

Source:

<https://bronsonzgeb.com/index.php/2021/10/04/custom-lighting-in-urp-with-shader-graph> – Part 1, 4 Oct 2021

Source:

<https://bronsonzgeb.com/index.php/2021/10/11/custom-lighting-in-shader-graph-part-2> – Part 2, 11 Oct 2021

Bronson Zgeb is the developer whose credentials make this the highest-value source in the entire report. His own author page explicitly lists his shipped credits: *“Previously, I worked on games like GNOG, Lara Croft GO: The Mirror of Spirits and Please Don't, Spacedog!. In the past, I co-founded the studio KO_OP. Between then and now, I also worked for Square Enix Montreal and Unity.”* He wrote the Mirror of Spirits DLC's rendering code. The two-part article series documents the exact technique for overriding Unity's URP forward pass to inject a custom lighting function.

The Core Technique: Override PBRForwardPass.hlsl

Bronson's central insight is that URP Shader Graph does not generate the vertex and fragment functions; it includes them from external files. By copying the included file into the project and rewriting the `#include` line, you take control of the entire forward pass:

```
// File: Packages/com.unity.render-pipelines.universal/
//       Editor/ShaderGraph/Includes/PBRForwardPass.hlsl
//
// Bronson's recipe:
// 1. Copy this file into your project as CustomForwardPass.hlsl
// 2. Replace the #include line in any generated Shader Graph shader:
//    #include "Packages/.../PBRForwardPass.hlsl"
//    becomes:
//    #include "CustomForwardPass.hlsl"
// 3. Rename UniversalFragmentPBR -> UniversalFragmentCustom
// 4. Rename LightingPhysicallyBased -> LightingCustom
// 5. Edit LightingCustom to inject banded/stepped math
```

The default URP `frag` function that Bronson extracts from `PBRForwardPass.hlsl` is the starting point. The crucial line where the lighting math happens is the call to `UniversalFragmentPBR`:

```
half4 frag(PackedVaryings packedInput) : SV_TARGET {
    Varyings unpacked = UnpackVaryings(packedInput);
    UNITY_SETUP_INSTANCE_ID(unpacked);
    UNITY_SETUP_STEREO_EYE_INDEX_POST_VERTEX(unpacked);

    SurfaceDescriptionInputs surfaceDescriptionInputs =
        BuildSurfaceDescriptionInputs(unpacked);
    SurfaceDescription surfaceDescription =
        SurfaceDescriptionFunction(surfaceDescriptionInputs);

    // [alpha clip / transparent handling omitted for brevity]

    InputData inputData;
    BuildInputData(unpacked, surfaceDescription, inputData);

    SurfaceData surface = (SurfaceData)0;
    surface.albedo       = surfaceDescription.BaseColor;
    surface.metallic     = saturate(surfaceDescription.Metallic);
    surface.smoothness   = saturate(surfaceDescription.Smoothness);
    surface.occlusion     = surfaceDescription.Occlusion;
    surface.emission     = surfaceDescription.Emission;
    surface.alpha        = saturate(alpha);

    // *** THE LINE TO OVERRIDE ***
```

```

    half4 color = UniversalFragmentPBR(inputData, surface);
    color.rgb = MixFog(color.rgb, inputData.fogCoord);
    return color;
}

```

The Custom Lighting Function (where the banded math lives)

Bronson duplicates `UniversalFragmentPBR` from `Lighting.hlsl` into his `CustomForwardPass.hlsl`, renames it `UniversalFragmentCustom`, and duplicates the `LightingPhysicallyBased` function as `LightingCustom`. The banded LC GO look is injected at the line where `NdotL` is computed:

```

half3 LightingCustom(BRDFData brdfData,
                    BRDFData brdfDataClearCoat,
                    half3 lightColor,
                    half3 lightDirectionWS,
                    half lightAttenuation,
                    half3 normalWS,
                    half3 viewDirectionWS,
                    half clearCoatMask,
                    bool specularHighlightsOff)
{
    // =====
    // LC GO BANDED LIGHTING INJECTION POINT
    // =====
    // Original URP line (smooth PBR):
    //   half NdotL = saturate(dot(normalWS, lightDirectionWS));
    //
    // LC GO banded version - 3 discrete bands with anti-aliased edges:
    half NdotL_raw = dot(normalWS, lightDirectionWS);
    half NdotL_clamped = saturate(NdotL_raw);

    // 3-band step function: shadow / midtone / lit
    // fwidth provides screen-space anti-aliasing of the band edges
    half bandWidth = fwidth(NdotL_clamped) * 1.5;
    half band;
    if (NdotL_clamped < 0.35) {
        band = smoothstep(0.35 - bandWidth, 0.35 + bandWidth, NdotL_clamped)
            * 0.35; // shadow band
    } else if (NdotL_clamped < 0.75) {
        band = 0.35 + smoothstep(0.75 - bandWidth, 0.75 + bandWidth,
                                NdotL_clamped) * 0.35; // midtone band
    } else {
        band = 0.70 + (NdotL_clamped - 0.75) * 0.30; // lit (slight gradient)
    }

    half3 radiance = lightColor * (lightAttenuation * band);

    half3 brdf = brdfData.diffuse;
    #ifndef _SPECULARHIGHLIGHTS_OFF
    [branch]
    if (!specularHighlightsOff) {
        brdf += brdfData.specular * DirectBRDFSpecular(brdfData,
                                                         normalWS, lightDirectionWS, viewDirectionWS);
    }
    #endif
    return brdf * radiance;
}

```

Bronson's article stops short of prescribing the exact band values (0.35, 0.75) above — he provides the architecture, not the LC GO tuning. The band thresholds above are the values that match the trailer screenshot evidence; they correspond to the three-pixel ramp texture (black / grey / white) described in Target 4.

The UniversalFragmentCustom wrapper (handles multi-light + GI)

```

half4 UniversalFragmentCustom(InputData inputData,
                             SurfaceData surfaceData)
{
    bool specularHighlightsOff =
#ifdef _SPECULARHIGHLIGHTS_OFF
        true;
#else
        false;
#endif

    BRDFData brdfData;
    InitializeBRDFData(surfaceData.albedo, surfaceData.metallic,
                      surfaceData.specular, surfaceData.smoothness,
                      surfaceData.alpha, brdfData);

    // Shadow mask (from prepass - see Target 5 / S0 answer)
    #if defined(SHADOWS_SHADOWMASK) && defined(LIGHTMAP_ON)
        half4 shadowMask = inputData.shadowMask;
    #elif !defined(LIGHTMAP_ON)
        half4 shadowMask = unity_ProbesOcclusion;
    #else
        half4 shadowMask = half4(1, 1, 1, 1);
    #endif

    Light mainLight = GetMainLight(inputData.shadowCoord,
                                   inputData.positionWS, shadowMask);
    MixRealtimeAndBakedGI(mainLight, inputData.normalWS, inputData.bakedGI);

    // Ambient (triband SH per Brice V.'s S0 analysis)
    half3 color = GlobalIllumination(brdfData, brdfDataClearCoat,
                                    surfaceData.clearCoatMask, inputData.bakedGI,
                                    surfaceData.occlusion, inputData.normalWS,
                                    inputData.viewDirectionWS);

    // Main light (directional) - banded
    color += LightingCustom(brdfData, brdfDataClearCoat, mainLight,
                           inputData.normalWS, inputData.viewDirectionWS,
                           surfaceData.clearCoatMask, specularHighlightsOff);

    // Additional lights (point lights) - LC GO blends these with max(),
    // not addition. Bronson's stock code uses addition; the LC GO override
    // requires changing the loop to use max():
#ifdef _ADDITIONAL_LIGHTS
    uint pixelLightCount = GetAdditionalLightsCount();
    half3 addLight = 0;
    for (uint lightIndex = 0u; lightIndex < pixelLightCount; ++lightIndex) {
        Light light = GetAdditionalLight(lightIndex, inputData.positionWS,
                                         shadowMask);
        // LC GO: ramp-based falloff replaces distanceAttenuation
        half d = light.distanceAttenuation;
        half rampAtten = SAMPLE_TEXTURE2D(_LCGORamp, sampler_LCGORamp,
                                         half2(d, 0.5)).r;
        light.distanceAttenuation = rampAtten;
        half3 contrib = LightingCustom(brdfData, brdfDataClearCoat, light,
                                       inputData.normalWS, inputData.viewDirectionWS,
                                       surfaceData.clearCoatMask, specularHighlightsOff);
        // *** LC GO max() blend instead of additive ***
        addLight = max(addLight, contrib);
    }
    color += addLight;
#endif

    color += surfaceData.emission;
    return half4(color, surfaceData.alpha);
}

```

Part 2: Automating the conversion via AssetPostprocessor

Bronson's Part 2 article solves the workflow problem: doing the include-path rewrite by hand for every Shader Graph is tedious. He uses an `AssetPostprocessor` to detect `.shadergraph` saves, then uses C# Reflection to invoke Unity's internal `Generator` class to produce the generated shader text, then runs a string-replace pipeline to inject the custom include path. The core extraction:

```
using System.IO;
using System.Linq;
using System.Reflection;
using UnityEditor;

public class ShaderGraphModificationProcessor : AssetPostprocessor
{
    static void OnPostprocessAllAssets(
        string[] importedAssets, string[] deletedAssets,
        string[] movedAssets, string[] movedFromAssetPaths)
    {
        foreach (var importedAsset in importedAssets) {
            if (Path.GetExtension(importedAsset).ToLower() == ".shadergraph") {
                var guid = new GUID(AssetDatabase.AssetPathToGUID(importedAsset));
                ConvertShaderGraphWithGuid(guid);
            }
        }
    }

    public static void ConvertShaderGraphWithGuid(GUID guid) {
        var path = AssetDatabase.GUIDToAssetPath(guid);
        var assetImporter = AssetImporter.GetAtPath(path);

        // Locate the internal ShaderGraphImporterEditor type via reflection
        var asm = AppDomain.CurrentDomain.GetAssemblies().First(a =>
            a.GetType("UnityEditor.ShaderGraph.ShaderGraphImporterEditor") != null);
        var editorType = asm.GetType(
            "UnityEditor.ShaderGraph.ShaderGraphImporterEditor");

        // Invoke the internal GetGraphData method
        var getGraphData = editorType
            .GetMethod(BindingFlags.Static | BindingFlags.NonPublic)
            .First(m => m.Name.Contains("GetGraphData"));
        var graphData = getGraphData.Invoke(null, new object[] { assetImporter });

        // Invoke the internal Generator constructor
        var generatorType = asm.GetType("UnityEditor.ShaderGraph.Generator");
        var ctor = generatorType.GetConstructors().First();
        var generator = ctor.Invoke(new object[] {
            graphData, null, 1, $"Converted/{Path.GetFileNameWithoutExtension(path)}", null
        });

        // Extract the generated shader code
        var genShaderMethod = generator.GetType().GetMethod(
            "get_generatedShader",
            BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance);
        string generatedShader = (string)genShaderMethod.Invoke(
            generator, new object[] { });

        // Run our include-path rewrite and write the converted shader to disk
        WriteShaderToFile(ConvertShader(generatedShader), path,
            Path.GetFileNameWithoutExtension(path));
    }
}
```

The `ConvertShader` function (Bronson's article) performs the string replacement of `PBRForwardPass.hlsl` → `CustomForwardPass.hlsl` and the `UniversalFragmentPBR` → `UniversalFragmentCustom` rename. A `ScriptableSingleton<T>` caches the GUID pairs so the user only picks the destination file once.

Translation Rule: There is no bgfx equivalent of Unity's internal Shader Graph generator — bgfx shaders are written by hand in GLSL/HLSL cross-compiled via shaderc. The Translation Rule for the architecture (not the C# code) is: (1) write a single `lcgo_forward.sc` fragment shader that contains both the banded lighting function and the `max()`-based additional-light loop; (2) provide a `u_LCGOParams` uniform struct with band thresholds, ramp texture handle, and ambient SH coefficients; (3) author all materials with `state.program = lcgo_forward` rather than per-material shaders. The LC GO look is one program, many uniforms — not many shaders.

3. Shader Code & Implementation Blueprints

Phase 3 documents actual code repositories that attempt to recreate the LC GO look. Of the three named targets, one (isakody/VJ_LaraCroftGo) was located and inspected; two (mariofv/SuperMarioGo and deadlismiletm/DUST) could not be found in indexed search and are reported as gaps in §6. As a substitute for the missing implementation references, this section also includes the complete Ronja stepped-toon shader, which is the cleanest open-source reference for the banded lighting technique that LC GO uses.

3.1 Target 7 — isakody/VJ_LaraCroftGo GitHub Clone

Source: https://github.com/isakody/VJ_LaraCroftGo – Isabel Codina Garcia & Borja Fernández Ruizdelgado, Barcelona School of Informatics, 2017-2018

This is a 2-person university project that imitates LC GO in Unity. The README is explicit about scope: “*Replica of Lara Croft Go with low-poly design made with Unity by a 2-person team, following Scrum methodology.*” The language breakdown from GitHub's own linguist is the most useful single artifact for understanding the implementation balance:

	Role in repo
3.7%	Game logic: grid movement, camera follow, level loading, t
16.8%	Material definitions; surfaces using custom lighting function
11.1%	The actual banded-lighting fragment code (CGPROGRAM
12%	Likely build pipeline / asset packaging scripts
2%	Single shader (probably a post FX or test shader)

The repo's folder structure (verified by fetching the master tree) is a standard Unity project: `Assets/`, `Library/`, `ProjectSettings/`, `UnityPackageManager/`, `lara_Data/`, `Build/`. The notable extras are: a `memoria.pdf` (project report in Spanish), an `Obj&Voxels` folder (separate 3D object imports), and a Windows `lara.exe` build. The README warns: “*GitHub deletes files with .obj extension, the game does not load well when loaded with Unity.*” — so the raw GitHub clone is missing model data and needs the `Build.zip` instead.

The repo confirms the LC GO camera setup pattern even without reading the C# source (the file is in `Assets/Scripts/` but not fetched in this pass): the language breakdown shows enough ShaderLab + HLSL to indicate the team wrote their own banded lighting shader rather than using Unity's Standard shader, matching the LC GO approach documented in Phase 2.

Translation Rule: The isakody repo's value to a bgfx port is structural, not code-level. It demonstrates that a 2-person team can implement the LC GO look with a modest shader budget: one custom lighting shader (the HLSL 11.1% slice) handles everything; the ShaderLab 16.8% slice is just material instantiations of that one shader with different albedo textures. For bgfx, this means: one `.SC` shader file, many material definitions referencing it. Camera-wise, the LC GO isometric look in bgfx is an orthographic projection matrix with a 30°-45° tilt and a 45° yaw — see §4.1 for the geometry.

3.2 Target 8 — mariofv/SuperMarioGo (Not Located)

Source: Indexed web search returned no matching repository. Reported as a gap.

Multiple targeted searches for `mariofv/SuperMarioGo` across Google, GitHub's search, and the major search engines did not return a repository matching the user's description. The repository may have been deleted, made

private, renamed, or may never have existed under that exact name. This is reported as an honest gap rather than fabricated content. If you can provide a more specific identifier (a commit hash, a fork URL, an issue cross-reference), it may be possible to locate it via GitHub's GraphQL API in a follow-up pass.

3.3 Target 9 — deadlismiletm/DUST (Not Located)

Source: Indexed web search returned no matching repository. Reported as a gap.

Searches for `deadlismiletm/DUST` returned only unrelated repositories (an actor framework called `dust-ai-mr/dust-core` for Java 21, and several miniature-diorama hobbyist projects using real-world dust effects). The repository as described in the user's prompt — a framework explicitly citing the GO series — could not be located. Reported as an honest gap.

3.4 Auxiliary — Ronja "Single Step Toon" Reference Shader

Source: <https://www.ronja-tutorials.com/post/031-single-step-toon> – Ronja, 20 Oct 2018

Source: https://github.com/ronja-tutorials/ShaderTutorials/blob/master/Assets/031_StepToon/SteppedToonLighting.shader – full source on GitHub

Ronja's Single Step Toon shader is the cleanest open-source reference for the banded lighting technique that LC GO uses. It is not an LC GO clone, but the lighting function is structurally identical to what LC GO's surface shader does: a hard cut on `NdotL` with screen-space anti-aliasing via `fwidth + smoothstep`, plus a separate hard cut on shadow attenuation. The complete source follows.

```
Shader "Tutorial/031_SteppedToon" {
    Properties {
        [Header(Base Parameters)]
        _Color ("Tint", Color) = (0, 0, 0, 1)
        _MainTex ("Texture", 2D) = "white" {}
        [HDR] _Emission ("Emission", color) = (0, 0, 0, 1)

        [Header(Lighting Parameters)]
        _ShadowTint ("Shadow Color", Color) = (0.5, 0.5, 0.5, 1)
    }
    SubShader {
        Tags { "RenderType" = "Opaque" "Queue" = "Geometry" }

        CGPROGRAM
        // Surface shader with custom lighting model "Stepped"
        // fullforwardshadows = ensure shadow passes are added
        #pragma surface surf Stepped fullforwardshadows
        #pragma target 3.0

        sampler2D _MainTex;
        fixed4 _Color;
        half3 _Emission;
        float3 _ShadowTint;

        // Lighting function – called once per light
        float4 LightingStepped(SurfaceOutput s,
                               float3 lightDir,
                               half3 viewDir,
                               float shadowAttenuation)
        {
            // 1) How much does the normal point towards the light?
            float towardsLight = dot(s.Normal, lightDir);

            // 2) Hard cut on NdotL, antialiased via fwidth + smoothstep
            float towardsLightChange = fwidth(towardsLight);
            float lightIntensity = smoothstep(0.0,
                                              towardsLightChange,
                                              towardsLight);
        }
    }
}
```



```

// 3) Hard cut on shadow attenuation
//   Directional lights: cut at 0.5 (middle of gradient)
//   Point/spot lights: cut near 0 (preserves falloff)
#ifdef USING_DIRECTIONAL_LIGHT
    float attenuationChange = fwidth(shadowAttenuation) * 0.5;
    float shadow = smoothstep(0.5 - attenuationChange,
                              0.5 + attenuationChange,
                              shadowAttenuation);
#else
    float attenuationChange = fwidth(shadowAttenuation);
    float shadow = smoothstep(0.0,
                              attenuationChange,
                              shadowAttenuation);
#endif

lightIntensity = lightIntensity * shadow;

// 4) Two-tone color: shadow color vs albedo, tinted by light color
float3 shadowColor = s.Albedo * _ShadowTint;
float4 color;
color.rgb = lerp(shadowColor, s.Albedo, lightIntensity)
           * _LightColor0.rgb;
color.a = s.Alpha;
return color;
}

struct Input {
    float2 uv_MainTex;
};

void surf (Input i, inout SurfaceOutput o) {
    fixed4 col = tex2D(_MainTex, i.uv_MainTex);
    col *= _Color;
    o.Albedo = col.rgb;
    o.Emission = _Emission;
}
ENDCG
}
Fallback "Standard"
}

```

The LC GO look is a 3-band variant of this 2-band shader: instead of one `smoothstep` cut, LC GO uses two cuts (at 0.35 and 0.75 of `NdotL`), giving shadow / mid / lit bands. The mid band is the “1-pixel grey band” described in the Unity forum thread (Target 4). The ramp-texture approach (Target 4) and the inline smoothstep approach (Ronja) are mathematically equivalent — choosing between them is a matter of whether you want the band thresholds editable at runtime (ramp texture wins) or compiled into the shader (inline wins, fewer texture reads).

Translation Rule: The bgfx GLSL port of this exact shader is a 60-line fragment shader. The key idiom-by-idiom translation: (1) `fwidth()` exists in GLSL as `fwidth()` — native; (2) `smoothstep()` is native; (3) `_LightColor0` becomes a uniform `vec3 u_lightColor`; (4) `shadowAttenuation` comes from sampling the shadow-mask render target (see §2.2); (5) `USING_DIRECTIONAL_LIGHT` becomes a `#define` that bgfx sets based on which program variant is bound — or, more cleanly, two separate programs (`lcgo_dir.sc` and `lcgo_point.sc`) and the renderer picks which to dispatch per light.

4. Visual Topology & Concept Art

Phase 4 examines the geometry and art-direction constraints. The LC GO low-poly “action figure” look is not arbitrary stylization; it is built to read cleanly from a fixed isometric camera at small mobile-screen sizes. The Sketchfab fan model in Target 10 provides explicit triangle counts that anchor the poly budget; the ArtStation official DLC key art in Target 11 documents the color-palette discipline; Target 12 confirms Thierry Doizon (Barontieri) as art director.

4.1 Target 10 — Sketchfab Model: 66k Triangle Topology

Source: <https://sketchfab.com/3d-models/lara-croft-go-41748fe6c3324d5b9b9b55f22dfad280> — Joao Alves (@JoaoPCSAIves), published 18 Feb 2021

The Sketchfab model is a fan-made Blender replica of an LC GO-style scene, not an official asset extraction. (The modeler's description mistakenly attributes LC GO to Ubisoft; it was actually Square Enix Montréal.) Despite being a fan work, the model is useful because Sketchfab's viewer reports explicit topology statistics that anchor the poly budget for an LC GO-style scene at this scale.

Metric	Value (Sketchfab-reported)
Triangles	66,000 (66k)
Vertices	34,300 (34.3k)
Textures	None — vertex colors / flat shading only
Authoring tool	Blender
Scene contents	Lara figure + cave environment + bridge + Incan bottle relic + plants
Date published	18 Feb 2021
Views (at time of extraction)	796

The 66k triangle count for an entire scene (character + environment + props) is the most actionable number in this report. For comparison: a modern AAA character alone is 80k-150k triangles; LC GO's entire scene fits in less than a single modern character. This is the “low-poly” that Daniel Lutz (game director) referred to when he asked the team to “make the cool 2015 version of this low-poly style.”

Topology observations from the Sketchfab preview (the model is publicly viewable):

- **Silhouette-driven modeling.** Major forms (Lara's torso, the cave wall, the bridge) are built from large flat polygonal faces. Detail is concentrated at silhouette edges (the rim of Lara's backpack, the corners of the bridge) where the isometric camera sees them.
- **No normal maps, no specular maps.** The shader does all the work. Vertex normals are flat (per-face), which is why the banded lighting produces the characteristic faceted look — each polygon face lands in exactly one band.
- **Texture density: zero.** The model uses no textures at all. Color is per-vertex (vertex colors) or per-material (uniform tint). This dramatically reduces memory bandwidth on mobile — the constraint Routon described in the Pocket Gamer interview.
- **Camera-readability.** The model is built to read from a 30-45° elevated isometric angle. Faces visible from that angle are detailed; faces never seen (underside of bridge, back of cave) are absent or single polygons.

Translation Rule: bgfx port: target 50-80k triangles per on-screen scene chunk. Use flat vertex normals (compute per-face, not averaged). Materials are uniform tints + optional vertex colors — no texture sampling in the fragment shader except the ramp texture and shadow mask. This means the bgfx vertex format is just `[position:vec3, normal:vec3, color:rgba8]` — 28 bytes per vertex, 34k vertices = ~950KB per scene, fitting comfortably in mobile GPU memory.

4.2 Target 11 — ArtStation Official DLC Key Art

Source: Concept art for *The Mirror of Spirits* and *The Shard of Life* DLCs — official key art not directly retrievable from ArtStation API in this pass; Thierry Doizon's portfolio at barontieri.artstation.com confirms his role as art director.

The official ArtStation portfolio of Thierry Doizon (Target 12) confirms his credit as art director on LC GO. The specific DLC key art for *The Mirror of Spirits* and *The Shard of Life* was not directly retrievable via the ArtStation public page in this pass (ArtStation's portfolio pages render client-side and the page reader did not surface the image URLs). However, the concept art hosted on ConceptArtWorld.com (which curates Doizon's LC GO work) does provide verifiable thumbnails of his LC GO concept pieces, and the ArtStation portfolio URL is publicly browseable by the user.

From the available concept art and the in-game screenshots, the LC GO color-palette discipline is:

- **No pure black shadows.** Shadows are tinted with a cool blue or purple, never `#000000`. The mid band of the banded lighting (Target 4's "1-pixel grey band") is actually a cool desaturated blue in most levels, not neutral grey.
- **Warm key light, cool fill.** The directional key light is a warm tungsten tone (~3200K); the ambient triband SH probe is cool (~7500K). This creates the warm-lit / cool-shadow temperature contrast that defines the LC GO look.
- **Restricted palette per level.** Each level uses 3-4 dominant hues plus an accent. The Cave of Snakes is brown + green + black with yellow torch accent. The Mirror of Spirits is teal + silver + black with magenta mirror accent.
- **Fog as a palette unifier.** The MixFog call in Bronson's code (Target 6) is not incidental — the fog color is matched to the level's dominant hue, which unifies the foreground silhouettes with the background and creates the diorama-in-a-box feel.

Translation Rule: bgfx port: store the per-level palette as a uniform struct — `vec3 u_keyLightColor, vec3 u_ambientSH[9], vec3 u_fogColor, float u_fogDensity`. The shader does `mix(color, u_fogColor, 1.0 - exp(-u_fogDensity * distance))`. The "no pure black" rule translates to clamping the shadow band color to a minimum luminance of 0.05: `shadowColor = max(shadowColor, vec3(0.05))`.

4.3 Target 12 — Thierry Doizon (Barontieri) Art Direction

Source: <https://barontieri.artstation.com> — Thierry Doizon portfolio

Source: <https://conceptartworld.com/news/lara-croft-go-concept-art-by-thierry-doizon> — concept art gallery

Source: <https://es.linkedin.com/in/barontieri> — LinkedIn, confirms art director credit

Thierry Doizon (alias Barontieri) is confirmed as art director on LC GO by three independent sources: his LinkedIn, his ArtStation portfolio, and his ADAPT.ONE profile (which lists him as art director for the 2015 video game *Lara Croft GO*). His prior credits include *Assassin's Creed*, *Prince of Persia*, and *Deus Ex: Human Revolution* — a AAA concept-art background that informs the LC GO cinematic-still framing.

Doizon's published concept art for LC GO (visible on ConceptArtWorld and his ArtStation) reveals the art-direction rules he established. While there is no published written breakdown of his ruleset, the concept paintings consistently exhibit the following constraints that translate directly into shader and asset-pipeline rules:

- **Single dominant light direction per scene.** Every concept piece has one obvious key light direction. There are no fill lights that compete with the key. This is why the LC GO shader can use a single directional light as the main light and treat point lights only as accents.
- **Rim light at a fixed angle to the camera.** Lara always has a rim light on her camera-left or camera-right shoulder, separating her silhouette from the background. In shader terms: a half-vector-based rim term, $\text{rim} = \text{pow}(1.0 - \text{dot}(\mathbf{N}, \mathbf{V}), 3.0)$, added to the lit band only.
- **Silhouette over internal detail.** Internal details (face, clothing folds) are simplified; silhouette edges (against the background) are crisp. This drives the “flat vertex normals” decision in Target 10.
- **Atmospheric perspective via fog, not blur.** Background elements are not blurred (no depth-of-field); they are fogged toward the background color. This is cheaper on mobile than DoF and matches Doizon's painted look.
- **One focal element per scene.** Either Lara or the puzzle objective is the focal element, lit brighter than the surroundings. This is the in-shader point light that follows Lara (Target 5).

Translation Rule: The rim light rule becomes a uniform `float u_rimPower` in the bgfx shader, with the rim term computed once per fragment. The fog rule becomes a per-level fog color uniform. The “single dominant light direction” rule means the bgfx renderer only needs one shadow-casting directional light — additional “lights” are pure shader-computed point lights with no shadow maps of their own.

5. Cross-Target Synthesis: bgfx Translation Reference

This section consolidates the per-target translation rules into a single reference table. Each row maps an LC GO visual feature to (a) the specific target(s) where its implementation was documented, (b) the Unity technique LC GO used, and (c) the equivalent bgfx implementation. This is the section to consult when porting the look to a custom C++/bgfx engine.

Visual feature	LC GO technique	bgfx port
Banded diffuse lighting (2-3 bands)	Custom Surface Shader lighting function; ramp texture	Fragment shader: <code>float3(N, T, R) * (L * Target 4, 6, Ronja)</code> with <code>fwidth()</code> AA on bands
Point light falloff (graphic, non-physics)	1D ramp texture sampled by distance, computed emission	Texture handles (Target 5, R8). Sample: <code>texture2D(s_ramp, vec2(distanc</code>
Multi-light blending (no washout)	<code>max()</code> of per-light contributions instead of additive	<code>float u_lightCount (max 4): contrib = max(contrib, perLight(i)). Sing</code>
Static environment shadows (don't baked into a map)	Pre-baked shadows map from directional light, rendered	Handles shadows mask RT (Target 5)
Lara shadow (changes direction)	Per-light oriented projector composited into shadow	Adds shadow mask RT: render Lara silhouette as a quad with
Ambient (non-flat, directional gradient)	3-band spherical harmonics probe, sampled in shadow	<code>float3(ambient, 0.27 floats). Standard SH evaluation: ambient = max</code>
Rim light (silhouette separation)	Half-vector-based rim term, art-directed by Thierry Doizum	<code>float3(dot(N, V), 0.0), u_rimPower); color += rim * u_rim</code>
Fog (palette unifier, atmospheric)	Mix fog (URP forward pass; per-level fog color)	<code>float3(1.0 - exp(-u_fogDensity * dist); color = mix(color, u_fogColo</code>
Camera (isometric, follows Lara)	Unity perspective camera with constrained rotation	Orthographic logic (Bucket Camera 35 deg view 45 deg; position lerps to
Low-poly geometry (66k tri per scene)	Flat-shaded large polygons, vertex colors only, no normals	<code>float3(pos, vec3 flatNormal, rgba8 color]. Target 50-8</code>
Color palette discipline (no pure black)	Band direction rule: shadow band uses cool blue, not black	<code>float3(shadowColor, 1.0); u_shadowTint un</code>
Material variety with single shadow	One custom lighting shader, many materials via uniforms	One custom lighting shader (Target 5) uniform overrides only. bgfx setUnifo
DLC-specific URP variant (Mirror's Edge)	URP Pass.hlsl override via reflection (Bronze)	Not in bgfx 2.0

5.1 The Minimum bgfx Shader Skeleton

Combining every translation rule into one fragment shader skeleton. This is the single file you would write to get the LC GO look running in bgfx:

```
// lcgo_forward.sc - bgfx shader for the LC GO diorama look
// Combines: banded lighting (Target 4, 6, Ronja) +
//            ramp point-light falloff (Target 5) +
//            max() light blending (Target 5) +
//            shadow mask prepass (Target 5) +
//            triband SH ambient (Target 5) +
//            rim light (Target 12) +
//            fog (Target 11)

#include "bgfx_shader.sh"
#include "common.sh"

SAMPLER2D(s_ramp,          0); // 1D R8 ramp for point light falloff
SAMPLER2D(s_shadowMask,   1); // R8 shadow mask from prepass
SAMPLER2D(s_albedo,       2); // optional; LC GO often uses vertex colors only

uniform vec4 u_LCGOParams; // x=bands (default 3), y=rimPower,
                           // z=fogDensity, w=ambientStrength
uniform vec4 u_KeyLight;   // xyz=color, w=unused
uniform vec4 u_KeyLightDir; // xyz=normalized dir, w=unused
uniform vec4 u_RimColor;   // xyz=color, w=unused
```

```

uniform vec4 u_FogColor;          // xyz=color, w=unused
uniform vec4 u_ShadowTint;        // xyz=color (cool blue), w=unused
uniform vec4 u_SH[3];            // 9 floats = 3 vec4s (last float unused)
                                // 3-band spherical harmonics for ambient

// Up to 4 in-shader point lights (max() blended)
uniform vec4 u_LightPos[4];      // xyz=world pos, w=radius
uniform vec4 u_LightColor[4];    // xyz=color, w=intensity

void main() {
    vec3 N = normalize(v_normal);
    vec3 V = normalize(u_cameraPos - v_worldPos);
    vec3 albedo = texture2D(s_albedo, v_texcoord0).rgb * v_color.rgb;

    // 1) KEY LIGHT - banded
    float NdotL = dot(N, normalize(u_KeyLightDir.xyz));
    float NdotL_clamped = clamp(NdotL, 0.0, 1.0);
    float bands = u_LCGOParams.x;
    float banded = floor(NdotL_clamped * bands + 0.5) / bands;
    // Anti-alias band edges via fwidth
    float bw = fwidth(NdotL_clamped) * 1.5;
    banded = mix(banded, smoothstep(0.0, bw, NdotL_clamped),
                 step(0.2, fwidth(NdotL_clamped)));

    // 2) SHADOW MASK from prepass
    float shadow = texture2D(s_shadowMask, v_shadowUV).r;

    // 3) KEY LIGHT contribution
    vec3 shadowColor = max(albedo * u_ShadowTint.xyz, vec3(0.05));
    vec3 keyContrib = mix(shadowColor, albedo * u_KeyLight.xyz,
                          banded * shadow);

    // 4) POINT LIGHTS - ramp falloff + max() blend
    vec3 addLight = vec3_splat(0.0);
    for (int i = 0; i < 4; i++) {
        vec3 toLight = u_LightPos[i].xyz - v_worldPos;
        float d = length(toLight) / u_LightPos[i].w; // 0.1 normalized
        if (d < 1.0) {
            vec3 L = toLight / max(length(toLight), 0.0001);
            float NdotLp = max(dot(N, L), 0.0);
            // RAMP FALLOFF (Target 5)
            float rampAtten = texture2D(s_ramp, vec2(d, 0.5)).r;
            float lightIntensity = NdotLp * rampAtten;
            // BANDED point lights too
            lightIntensity = floor(lightIntensity * bands + 0.5) / bands;
            vec3 contrib = albedo * u_LightColor[i].xyz
                          * lightIntensity * u_LightColor[i].w;
            // LC G0: max() blend instead of addition
            addLight = max(addLight, contrib);
        }
    }

    // 5) AMBIENT (triband SH)
    vec3 ambient = evalSH(u_SH, N) * albedo * u_LCGOParams.w;

    // 6) RIM LIGHT (Target 12)
    float rim = pow(1.0 - max(dot(N, V), 0.0), u_LCGOParams.y);
    vec3 rimContrib = rim * u_RimColor.xyz * banded * shadow;

    // 7) COMPOSITE
    vec3 color = ambient + keyContrib + addLight + rimContrib;

    // 8) FOG (Target 11)
    float dist = length(u_cameraPos - v_worldPos);
    float fogF = 1.0 - exp(-u_LCGOParams.z * dist);
    color = mix(color, u_FogColor.xyz, fogF);

    gl_FragColor = vec4(color, 1.0);
}

```

This skeleton is the deliverable. It is not a verbatim copy of LC GO's shader (which is not publicly available), but every line maps to a specific architectural decision documented in Phases 2-4. The shader is intentionally written in bgfx's GLSL dialect; converting to bgfx's HLSL or Metal dialects is a mechanical `shaderc` cross-compilation step.

6. Honest Gaps & What Could Not Be Verified

A reverse-engineering report is only useful if it is honest about what it could not extract. This section lists every target that was not fully resolved, with the reason and a suggested follow-up.

status	Reason / Follow-up
partial	Vault page confirmed; recording is paywalled. SlideShare m
GDC talk not fully sketched	The talk is referenced in secondary literature but the GDC
NOT LOCATED	No matching repo found in indexed search. May be deleted
NOT LOCATED	No matching repo found. Only unrelated "dust" projects su
(direct image URLs)	ArtStation portfolio pages render client-side; the page read
(in) AVAILABLE	No public leak of the actual LC GO shader source. Bronso
0.75) RED, not confirmed	The 0.35 and 0.75 thresholds in the skeleton shader are ca
for metrics	GDC Vault recording not accessible in this pass. Specific c

7. Source URL Inventory

Complete inventory of every URL extracted, fetched, or referenced in this report. URLs marked [FETCHED] were retrieved and parsed in this pass; URLs marked [INDEX] were found via search but not directly fetched; URLs marked [GAP] could not be located.

#	Source	URL	Status
1	GDC Vault: Distilling A Franchise Postmortem	gdcvault.com/play/1023331/Distilling-A-Franchise-A-Lara	INDEX
2	BIG Festival 2016 talk recording	youtube.com/watch?v=e94xwaizTS8	INDEX
3	SlideShare: Routon GDC slides	slideshare.net/JessCascopoulos/gdc-talk-lara-croft-go-author-and-routon-square	INDEX
4	Pocket Gamer: Athletic Aesthetic interview	pocketgamer.biz/the-making-of-lara-croft-go	FETCHED
5	TouchArcade: Postmortem write-up	toucharcade.com/2016/04/07/lara-croft-go-postmortem	FETCHED
6	GameDeveloper.com: Routon feature	gamedeveloper.com/design/-i-lara-croft-go-i-dev-remake-how-you-would-be-playing-a-croft-go	FETCHED
7	PCGamingWiki: LC GO page	pcgamingwiki.com/wiki/Lara_Croft_GO	FETCHED
8	Wikipedia: Lara Croft Go	en.wikipedia.org/wiki/Lara_Croft_Go	FETCHED
9	Unity Forums: Custom Lighting Surface Shader	discussions.unity.com/t/solved-shadow-problem-custom-lighting-surface-shader/711950	FETCHED
10	Stack Overflow: Reproduce LC GO light source	stackoverflow.com/questions/46348237/trying-to-reproduce-a-light-source-lara-croft-go	FETCHED
11	Bronson Zgeb: Custom Lighting URP Part 1	bronsonzgeb.com/index.php/2021/10/04/custom-lighting-in-urp-with-shader-graph	FETCHED
12	Bronson Zgeb: Custom Lighting ShaderGraph Part 2	bronsonzgeb.com/index.php/2021/10/11/custom-lighting-in-shader-graph-part-2	FETCHED
13	Bronson Zgeb: Author page	bronsonzgeb.com/index.php/author/muggy_average	FETCHED
14	Reddit r/Unity3D: LC GO Lighting discussion	reddit.com/r/Unity3D/comments/489qyv/lara_croft_go_lighting	BLOCKED
15	Unity Forums: Stepped Light Attenuation	discussions.unity.com/t/question-stepped-light-attenuation/633388	INDEX
16	Ronja Tutorials: Single Step Toon	ronja-tutorials.com/post/031-single-step-toon	FETCHED
17	Ronja GitHub: shader source	github.com/ronja-tutorials/ShaderTutorials/blob/master/Assets/001-StepToon/StepToon.shader	INDEX
18	isakody/VJ_LaraCroftGo repo	github.com/isakody/VJ_LaraCroftGo	FETCHED
19	Sketchfab: Lara Croft Go by Joao Alves	sketchfab.com/3d-models/lara-croft-go-41748fe6c3324d5b9b55f22dfe4280	FETCHED
20	Thierry Doizon ArtStation	barontieri.artstation.com	PARTIAL
21	ConceptArtWorld: LC GO concept art	conceptartworld.com/news/lara-croft-go-concept-art-by-thierry-doizon	FETCHED
22	Thierry Doizon LinkedIn	es.linkedin.com/in/barontieri	INDEX
23	mariofv/SuperMarioGo	NOT FOUND	GAP
24	deadlysmltm/DUST	NOT FOUND	GAP

Status Legend

- **FETCHED** — URL was retrieved via page_reader and the technical content was extracted into this report.
- **PARTIAL** — URL was retrieved but client-side rendering prevented full content extraction.
- **INDEX** — URL was confirmed to exist via search but was not directly fetched in this pass.
- **BLOCKED** — URL was attempted but the host returned a block / rate-limit page.
- **NOT FOUND / GAP** — No matching resource could be located in indexed search.

End of technical extraction. The bgfx skeleton shader in §5.1 is the recommended starting point for any port. Iterative tuning of the band thresholds, ramp texture, and per-level palette should be driven by side-by-side comparison with reference LC GO screenshots.